

Estrutura de Dados 1

Estruturas - Structs

- *struct* é uma abreviação para structure.
- É uma estrutura de dados composta que define fisicamente uma lista de variáveis agrupadas sob um nome em um bloco de memória.



```
struct [tag da estrutura] {  
    tipo_da_variavel nome_da_variavel;  
    tipo_da_variavel nome_da_variavel;  
    ...  
    tipo_da_variavel nome_da_variavel;  
} [uma ou mais variáveis da estrutura];
```



```
struct Livros {  
    char titulo[50];  
    char autor[50];  
    char assunto[100];  
    int id_livro;  
} livro;
```



```
struct Livros {  
    char titulo[50];  
    char autor[50];  
    char assunto[100];  
    int id_livro;  
};  
/* Declarando Livro1 e Livro2 do tipo Livros */  
struct Livros Livro1, Livro2;
```



```
struct Livros {  
    char titulo[50];  
    char autor[50];  
    char assunto[100];  
    int id_livro;  
} Livro1, Livro2;
```

// declarando as variáveis na própria estrutura, não é muito recomendável



```
struct Livros {  
    char titulo[50];  
    char autor[50];  
    char assunto[100];  
    int id_livro;  
};
```

// inicializando

```
struct Livros Livro1 = { "Título genérico", "Blog Trybe",  
    "Um livro bem genérico", 6495407 };
```



```
struct Livros {  
    char titulo[50];  
    char autor[50];  
    char assunto[100];  
    int id_livro;  
};  
/* Declarando Livro1 do tipo Livro */  
struct Livros Livro1;  
strcpy( Livro1.titulo, "Título genérico"); // inicializando os valores  
para cada variável separada  
strcpy( Livro1.autor, "Blog Trybe");  
strcpy( Livro1.assunto, "Um livro bem genérico");  
Livro1.id_livro = 6495407;
```



```
/* mostrando as informações do livro 1 */  
printf( "Livro 1 titulo : %s\n", Livro1.titulo);  
printf( "Livro 1 autor : %s\n", Livro1.autor);  
printf( "Livro 1 assunto : %s\n", Livro1.assunto);  
printf( "Livro 1 id_livro : %d\n", Livro1.id_livro);
```

```
/* mostrando as informações do livro 2 */  
printf( "Livro 2 titulo : %s\n", Livro2.titulo);  
printf( "Livro 2 autor : %s\n", Livro2.autor);  
printf( "Livro 2 assunto : %s\n", Livro2.assunto);  
printf( "Livro 2 id_livro : %d\n", Livro2.id_livro);
```



typedef: é usado para renomear um tipo de dado.

```
// C program to demonstrate typedef  
#include <stdio.h>
```

```
// After this line BYTE can be used  
// in place of unsigned char  
typedef unsigned char BYTE;
```

```
int main()  
{  
    BYTE b1, b2;  
    b1 = 'c';  
    printf("%c ", b1);  
    return 0;  
}
```



Outro exemplo:

Declare uma variável ponto (que é um par ordenado de inteiros). Preceda a declaração de um typedef:

```
typedef struct { int x; int y; } ponto;
```

Agora ponto passa a ser o nome de um novo tipo (idêntico a par ordenado de inteiros). Esse tipo pode ser usado para declarar novos pontos:

ponto a, b;

- O **typedef** é usado para criar um apelido (nome alternativo) para tipos de dados existentes, facilitando o uso de estruturas no código.
- No exemplo, o typedef foi utilizado para criar um novo tipo de dado chamado ponto, que é uma estrutura contendo dois inteiros (x e y), representando um par ordenado de números.
- Isso torna o código mais legível e modular, permitindo o uso de ponto como tipo para declarar variáveis que armazenam pontos no plano cartesiano.

```
ponto a, b;  
a.x = 1;  
a.y = 2;
```

```
b.x = 3;  
b.y = 4;
```

```
printf("Ponto A: (%d, %d)\n", a.x, a.y);  
printf("Ponto B: (%d, %d)\n", b.x, b.y);
```



```
typedef struct
{
    char nome[100];
    int idade;
} pessoa;

main()
{
    // alocando uma estrutura
    pessoa *p = (pessoa*) malloc(sizeof(pessoa));

    if (p)
    {
        p->idade = 3;
        printf("%d\n", p->idade);
        free(p);
    }
}
```

typedef é usado para criar um **apelido** para a estrutura, de modo que podemos utilizar `pessoa` em vez de `struct pessoa` ao declarar variáveis desse tipo.

Posteriormente, o programa aloca dinamicamente **memória para um único objeto do tipo `pessoa`** usando `malloc`.

- `malloc(sizeof(pessoa))`: Aloca memória suficiente para armazenar uma variável do tipo `pessoa` (100 bytes para o `nome` + 4 bytes para o `idade`, normalmente).
- `(pessoa*)` faz um **cast** do ponteiro retornado por `malloc` para o tipo `pessoa*`, que é o tipo de dado adequado para a variável `p`.

Verificação: A alocação de memória é verificada com `if (p)`, ou seja, verifica se `malloc` retornou um ponteiro válido (não NULL).

Atribuição e impressão: Se a memória foi alocada com sucesso, o código:

Atribui 3 à variável `idade` da estrutura `p` (`p->idade = 3`).
Imprime a idade usando `printf` (`printf("%d\n", p->idade)`), que deverá exibir 3.

Liberação da memória: Após o uso da memória alocada, `free(p)` é chamado para liberar a memória previamente alocada.

```
typedef struct
```

```
{
```

```
float Peso; // define o campo Peso
```

```
int Idade; // define o campo Idade
```

```
float Altura; // define o campo Altura
```

```
} Pessoa; // Define o nome do novo tipo criado
```

Após a criação do tipo, é possível declarar variáveis do tipo Pessoa, desta forma:

```
Pessoa Joao, P1, P2;
```

```
Pessoa Povo[10]; // cria um vetor de 10 pessoas.
```

No caso, as variáveis **Joao**, **P1** e **P2** são todas do tipo Pessoa. Cada uma dessas variáveis armazenará informações sobre uma pessoa, como o peso, a idade e a altura.

`Pessoa Povo[10];`. Esse vetor pode armazenar 10 diferentes pessoas, cada uma com seus respectivos dados.



Conjuntos de Estruturas

Para acessar os campos de uma struct, usa-se a sintaxe **NomeDaVariavel.NomeDoCampo**, conforme o exemplo a seguir.

```
Joao.Idade = 15;  
Joao.Peso = 60.5;  
Joao.Altura = 1.75;
```

```
Povo[4].Idade = 23; // Atribui 23 à idade da 5ª pessoa (índice 4) no vetor Povo.  
Povo[4].Peso = 75.3; // Atribui 75.3 ao peso da 5ª pessoa no vetor Povo.  
Povo[4].Altura = 1.89; // Atribui 1.89 à altura da 5ª pessoa no vetor Povo.
```

Outra vantagem de utilizar struct é a possibilidade de atribuir os dados de uma struct para outra, com apenas um comando de atribuição, como neste exemplo:

```
P2 = Povo[4]; // Atribui os dados da 5ª pessoa (Povo[4]) para P2
```



```
#include <stdio.h>
```

```
typedef struct // Cria uma STRUCT para armazenar os dados de uma pessoa
```

```
{  
    float Peso; // define o campo Peso  
    int Idade; // define o campo Idade  
    float Altura; // define o campo Altura  
} Pessoa; // Define o nome do novo tipo criado
```

```
void ImprimePessoa(Pessoa P) // declara o parâmetro como uma struct
```

```
{  
    printf("Idade: %d Peso: %f Altura: %f\n", P.Idade, P.Peso, P.Altura);  
}
```



Estruturas como Parâmetros

```
int main()
{
    Pessoa Joao, P2;
    Pessoa Povo[10];

    Joao.Idade = 15;
    Joao.Peso = 60.5;
    Joao.Altura = 1.75;

    Povo[4].Idade = 23;
    Povo[4].Peso = 75.3;
    Povo[4].Altura = 1.89;

    P2 = Povo[4];
    P2.Idade++;

    // chama a função que recebe a struct como parâmetro
    ImprimePessoa(Joao);
    ImprimePessoa(Povo[4]);
    ImprimePessoa(P2);
    return 0;
}
```

• **ImprimePessoa (Joao)** : Imprime os dados de **Joao**.
• **ImprimePessoa (Povo [4])** : Imprime os dados da 5ª pessoa no vetor **Povo**.
• **ImprimePessoa (P2)** : Imprime os dados da pessoa **P2** (que agora tem a idade incrementada).
Idade: 15 Peso: 60.50 Altura: 1.75
Idade: 23 Peso: 75.30 Altura: 1.89
Idade: 24 Peso: 75.30 Altura: 1.89



Retorno de Structs em Funções

```
#include <stdio.h>
```

```
typedef struct // Cria uma STRUCT para armazenar os dados de uma pessoa  
{  
    float Peso; // define o campo Peso  
    int Idade; // define o campo Idade  
    float Altura; // define o campo Altura  
} Pessoa; // Define o nome do novo tipo criado
```

```
Pessoa SetPessoa(int idade, float peso, float altura)  
{  
    Pessoa P;  
    P.Idade = idade;  
    P.Peso = peso;  
    P.Altura = altura;  
    return P; // retorna a struct contendo os dados passados por parâmetro  
}
```



Retorno de Structs em Funções

```
void ImprimePessoa(Pessoa P) // declara o parâmetro como uma struct  
{  
    printf("Idade: %d Peso: %f Altura: %f\n", P.Idade, P.Peso, P.Altura);  
}
```

```
int main()  
{  
    Pessoa Joao;  
    // atribui o retorno da função a uma variável struct:  
    Joao = SetPessoa(15,60.5,1.75);  
    ImprimePessoa(Joao);  
    return 0;  
}
```



```
#include <stdio.h>
```

```
typedef struct // Cria uma STRUCT para armazenar os dados de uma pessoa
{
    float Peso; // define o campo Peso
    int Idade; // define o campo Idade
    float Altura; // define o campo Altura
} Pessoa; // Define o nome do novo tipo criado

void ImprimePessoa(Pessoa P)
{
    printf("Idade: %d Peso: %f Altura: %f\n", P.Idade, P.Peso, P.Altura);
}
```



Para passar uma struct por referência, deve-se passar um ponteiro para a struct, como no exemplo a seguir.

```
// Nesta função o parâmetro P é um ponteiro para uma struct
void SetPessoa(Pessoa *P, int idade, float peso, float altura)
{
    (*P).Idade = idade; // o campo pode ser acessado desta forma
    P->Peso = peso;      // ou desta
    P->Altura = altura;
}

int main()
{
    Pessoa Joao;
    SetPessoa(&Joao, 15, 70.5, 1.75);
    ImprimePessoa(Joao);
    return 0;
}
```

Parâmetro Pessoa *P: A função SetPessoa recebe um **ponteiro** para uma variável do tipo Pessoa (Pessoa *P). Isso significa que a função vai modificar diretamente a variável Pessoa passada como argumento.

(*P).Idade = idade;

Esta linha utiliza a **notação de desreferência** para acessar e modificar o campo Idade da estrutura Pessoa apontada pelo ponteiro P. (*P) desreferencia o ponteiro P, acessando a estrutura Pessoa apontada por ele. Depois, (*P).Idade acessa o campo Idade dessa estrutura.

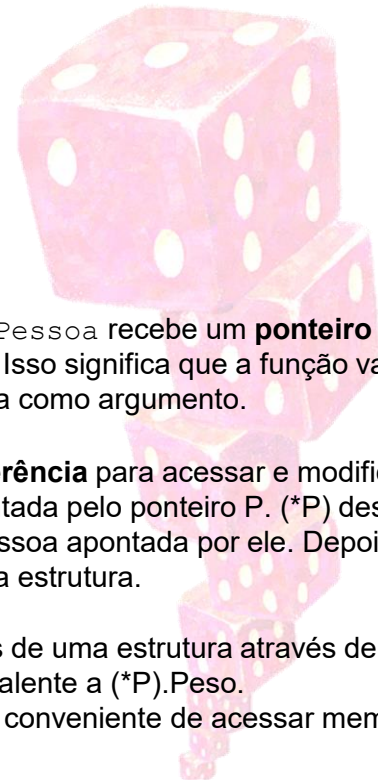
P->Peso = peso;

Este é um **atalho** para acessar campos de uma estrutura através de um ponteiro. A expressão P->Peso é equivalente a (*P).Peso.

A seta (->) é uma forma mais comum e conveniente de acessar membros de uma estrutura através de um ponteiro.

P->Altura = altura;

Semelhante a P->Peso, essa linha define o campo Altura da estrutura apontada por P.



<https://www.embarcados.com.br/struct-registros-em-linguagem-c/>

<https://www.inf.pucrs.br/~pinho/Laprol/Structs/Structs.htm>

<http://linguagemc.com.br/alocacao-dinamica-de-memoria-em-c/>

<http://www.univasf.edu.br/~criston.souza/algoritmos/arquivos/aula14.pdf>

<https://www.ime.usp.br/~pf/algoritmos/aulas/footnotes/typedef.html>

<https://www.geeksforgeeks.org/typedef-versus-define-c/>



Exercício 1: Definindo uma Struct

1.Objetivo: Definir uma struct para armazenar as informações de um livro.

2.Descrição:

1. A struct deverá conter os seguintes campos:

1. Título: String de 100 caracteres

2. Autor: String de 50 caracteres

3. Ano de publicação: Inteiro

4. Número de páginas: Inteiro

3.Tarefa: Escreva o código para a definição da struct e crie duas variáveis dessa struct para armazenar informações de dois livros diferentes. Mostre como você inicializaria os dados para essas duas variáveis.



Exercício 2: Acessando os Campos de uma Struct

Objetivo: Acessar os campos de uma struct e modificar seus valores.

Descrição:

Suponha que você tenha uma struct chamada Pessoa, com os campos:

Nome: String de 50 caracteres

Idade: Inteiro

Altura: Float

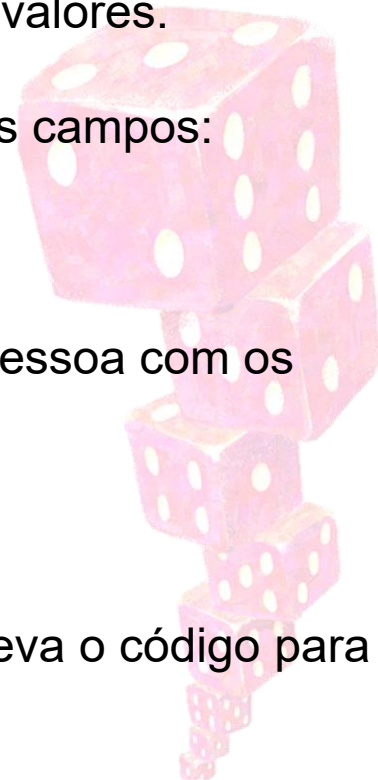
Tarefa: Escreva o código que inicializa uma variável do tipo Pessoa com os seguintes dados:

Nome: "João"

Idade: 25

Altura: 1.80

Depois, modifique a idade para 26 e a altura para 1.81 e escreva o código para imprimir as novas informações.



Exercício 3: Usando Structs em Funções

Objetivo: Passar uma struct como parâmetro para uma função.

Descrição:

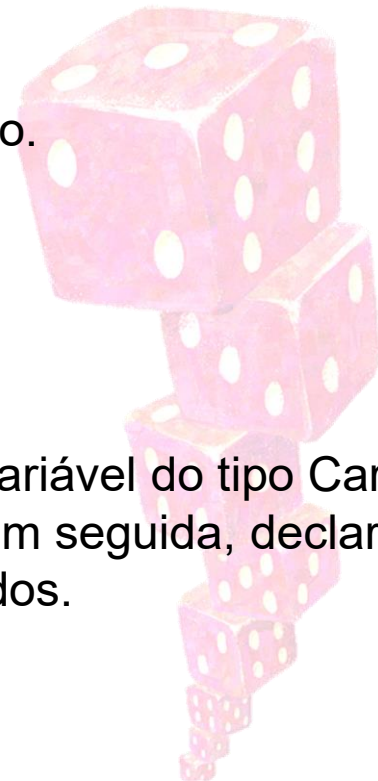
Crie uma struct chamada Carro com os campos:

Modelo: String de 50 caracteres

Ano de fabricação: Inteiro

Cor: String de 30 caracteres

Tarefa: Escreva uma função ImprimirCarro que recebe uma variável do tipo Carro como parâmetro e imprime as informações do carro na tela. Em seguida, declare uma variável Carro e chame essa função para imprimir os dados.



Exercício 4: Passando uma Struct por Referência

Objetivo: Modificar os dados de uma struct dentro de uma função, utilizando passagem por referência.

Descrição:

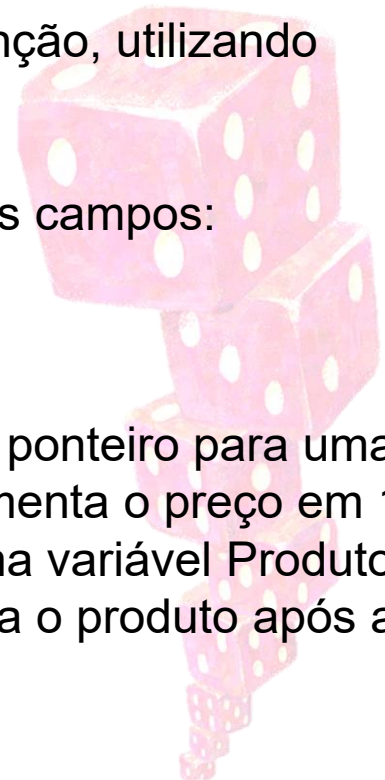
Suponha que você tenha uma struct Produto com os seguintes campos:

Nome: String de 50 caracteres

Preço: Float

Quantidade em estoque: Inteiro

Tarefa: Escreva uma função AtualizarProduto que recebe um ponteiro para uma struct Produto e altera os dados do produto (por exemplo, aumenta o preço em 10% e diminui a quantidade em estoque em 1). Depois, declare uma variável Produto, chame a função para modificar os dados e mostre como ficaria o produto após a atualização.



Exercício 5: Matrizes de Structs

Objetivo: Trabalhar com arrays de structs.

Descrição:

Crie uma struct Aluno com os campos:

Nome: String de 50 caracteres

Nota1: Float

Nota2: Float

Nota3: Float

Tarefa: Declare um array de 3 alunos e inicialize os dados de cada aluno manualmente. Em seguida, escreva um código que calcule a média das notas de cada aluno e imprima os nomes dos alunos com suas respectivas médias.

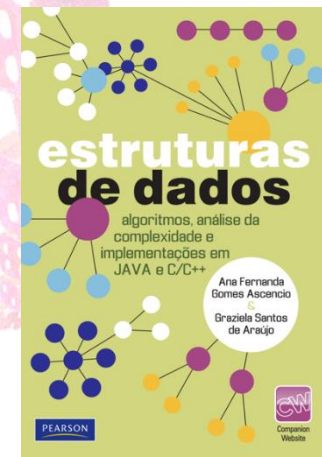


Livro 1: C: Como Programar (Deitel e Deitel, 2011):

- (a) **Tópico:** Linguagem C: Capítulos 2 a 9
- (b) **Tópico:** Organização de Arquivos: Capítulo 11
- (c) **Tópico:** Ponteiros e Alocação Dinâmica: Capítulo 7
- (d) **Tópico:** Recursividade: Seção 5.14
- (e) **Exemplos:** <https://github.com/pdeitel/CHowtoProgram9e/tree/master/examples>

Livro 2: Estrutura de Dados: Algoritmos, Análise da Complexidade e Implementações em Java e C/C++ (Ascencio e Araújo, 2010):

- (a) **Tópico:** Introdução à Complexidade Computacional: Capítulo 1
- (b) **Tópico:** Algoritmos de Busca: Capítulo 2
- (c) **Tópico:** Algoritmos de Ordenação: Capítulo 2
- (d) **Tópico:** Estruturas de Dados Lineares: Capítulos 3 e 4
- (e) **Tópico:** Algoritmos em Árvores Binárias: Capítulo 7



Algorithms in C, Parts 1-4: Fundamentals, Data Structures, Sorting, Searching (3ª edição) de Robert Sedgewick

Sumário de Capítulos (Parts 1–4)

I. FUNDAMENTALS — Fundamentos

Introduction

Algoritmos

Problema de exemplo (Connectivity, Union-Find)

Resumo dos tópicos

Principles of Algorithm Analysis

Implementação e análise empírica

Crescimento de funções

Notação Big-Oh e recursões

II. DATA STRUCTURES — Estruturas de Dados

Elementary Data Structures

Arrays

Listas ligadas

Strings

Estruturas compostas

Abstract Data Types (ADTs)

Pilhas (Stack), Filas (Queues)

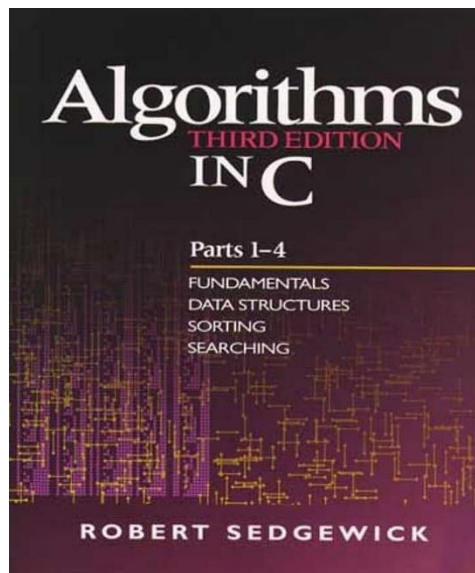
Diferença entre ADT e implementações

Recursion and Trees

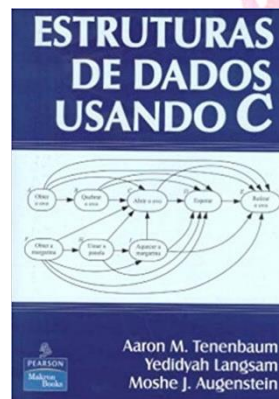
Algoritmos recursivos

Árvores e suas propriedades

Traversal e algoritmos de árvore



Estruturas de Dados e Algoritmos em C
(Aaron M. Tenenbaum,
Mosab M. J. Abdallah),
Pearson, 1995



Drozdek Ferraz, I. N.
(2003). Programação
com Arquivos. Manole.